

Relatório fase 1 CSS

Ravi Abreu 59835

Pedro Silva 59870

Luís Colaço 59834

Mapeamento JPA e Decisões Tomadas

Utilizador, Árbitro e Jogador

No projeto, a entidade Utilizador foi definida como classe abstrata, da qual derivam as classes Jogador e Arbitro. As três classes foram anotadas como @Entity e optou-se pela estratégia de herança Table per Class (TPC), onde cada subclasse tem a sua própria tabela com os seus atributos e os herdados da superclasse. Esta estratégia da herança foi escolhida porque um jogador não pode ser árbitro, e vice-versa.

Validação de Dados : @Column(nullable = false) garante que campos obrigatórios sejam preenchidos, evitando inconsistências no banco de dados. @Column(unique = true): Evita duplicação de dados críticos, como usernames.

A classe Jogador representa os jogadores no sistema e herda da classe Utilizador. Ela adiciona atributos específicos para jogadores, como posição, equipas, cartões recebidos e jogos.

- Relacionamento com Equipa
Anotação: @ManyToMany(mappedBy = "jogadores")
Justificativa: Um jogador pode pertencer a várias equipas, e uma equipa pode ter vários jogadores.
- Relacionamento com Cartao
Anotação: @OneToMany(mappedBy = "jogador")
Justificativa: Um jogador pode receber vários cartões durante os jogos.
- Relacionamento com Jogo
Anotação: @ManyToMany
Justificativa: Um jogador pode participar de vários jogos, e um jogo pode ter vários jogadores.

A classe Arbitro representa os árbitros no sistema e herda da classe Utilizador. Ela adiciona atributos específicos para árbitros, como certificação e jogos oficiados.

- Relacionamento com Jogo
Anotação: @ManyToMany(mappedBy = "arbitros")
Justificativa: Um árbitro pode officiar vários jogos, e um jogo pode ter vários árbitros.

Equipa

Equipa representa as equipas no sistema e foi anotada como uma entidade JPA com @Entity. Esta classe contém atributos específicos para equipas, como nome(único), jogadores, jogos (casa e visitante) e conquistas.

Validação de Dados: o atributo nome é obrigatório, garantindo que nenhuma equipa seja criada sem um nome válido e único. A validação de relacionamentos impede que equipas sejam associadas a jogadores ou jogos inexistentes.

- Relacionamento com Jogador
Anotação: @ManyToMany
Justificativa: Uma equipa pode ter vários jogadores, e um jogador pode pertencer a várias equipas (ex.: seleções e clubes).
- Relacionamento com Jogo
Anotação: @OneToMany(mappedBy = "equipaCasa") para jogos em que a equipa é a equipa da casa.
@OneToMany(mappedBy = "equipaVisitante") para jogos em que a equipa é a equipa visitante.
Justificativa: Uma equipa pode participar de vários jogos, tanto como equipa da casa quanto como equipa visitante.
- Relacionamento com Conquista
Anotação: @ElementCollection
Justificativa: O uso de @ElementCollection é apropriado porque as conquistas não são entidades independentes, mas sim informações associadas diretamente à equipa.

Jogo

A classe Jogo representa os jogos no sistema e foi anotada como uma entidade JPA com @Entity. Esta classe contém atributos essenciais para descrever um jogo, como data, hora, local, equipas participantes, árbitros e estatísticas.

A Estratégia de Herança foi: @Inheritance(strategy = InheritanceType.SINGLE_TABLE) pois a estratégia SINGLE_TABLE armazena todas as subclasses de Jogo em uma única tabela, uma vez que as entidades que subclasses de jogo (JogoCampeonato e JogoAmigavel) são bastante similares, o que não provocará muitas colunas a null, tornando efetiva esta estratégia.

Validação de Dados: os atributos obrigatórios (data, hora, local, equipaCasa, equipaVisitante) são validados pelo negócio, impedindo a criação de jogos com dados incompletos.

- Relacionamento com Equipa
Anotação: @ManyToOne para equipaCasa e equipaVisitante.
Justificativa: Um jogo envolve duas equipas: uma equipa da casa e uma equipa visitante.
- Relacionamento com Equipa
Anotação: @ManyToOne para equipaCasa e equipaVisitante.
Justificativa: Um jogo envolve duas equipas: uma equipa da casa e uma equipa visitante.
- Relacionamento com EstatisticasJogo
Anotação: @OneToOne(cascade = CascadeType.ALL, fetch = FetchType.EAGER)
Justificativa: Cada jogo possui um conjunto único de estatísticas, como marcadores e cartões.

O mapeamento @OneToOne reflete essa relação de exclusividade. O uso de cascade = CascadeType.ALL garante que as estatísticas sejam criadas, atualizadas ou excluídas automaticamente junto com o jogo. O fetch = FetchType.EAGER garante que as estatísticas sejam carregadas automaticamente ao buscar um jogo.